Listing 12.13 A more flexible implementation of `zip`

```
def zipL[A,B](fa: F[A], fb: F[B]): F[(A, Option[B])] =
  (mapAccum(fa, toList(fb)) {
    case (a, Nil) => ((a, None), Nil)
    case (a, b :: bs) => ((a, Some(b)), bs)
  })._1

def zipR[A,B](fa: F[A], fb: F[B]): F[(Option[A], B)] =
  (mapAccum(fb, toList(fa)) {
    case (b, Nil) => ((None, b), Nil)
    case (b, a :: as) => ((Some(a), b), as)
  })._1
```

These implementations work out nicely for `List` and other sequence types. In the case of `List`, for example, the result of `zipR` will have the shape of the `fb` argument, and it will be padded with `None` on the left if `fb` is longer than `fa`.

For types with more interesting structures, like `Tree`, these implementations may not be what we want. Note that in `zipL`, we're simply flattening the right argument to a `List[B]` and discarding its structure. For `Tree`, this will amount to a preorder traversal of the labels at each node. We're then "zipping" this sequence of labels with the values of our left `Tree`, `fa`; we aren't skipping over nonmatching subtrees. For trees, `zipL` and `zipR` are most useful if we happen to know that both trees share the same shape.

12.7.4 Traversal fusion

In chapter 5, we talked about how multiple passes over a structure can be fused into one. In chapter 10, we looked at how we can use monoid products to carry out multiple computations over a foldable structure in a single pass. Using products of applicative functors, we can likewise fuse multiple traversals of a traversable structure.

EXERCISE 12.18

Use applicative functor products to write the fusion of two traversals. This function will, given two functions `f` and `g`, traverse `fa` a single time, collecting the results of both functions at once.

```
def fuse[G[_],H[_],A,B](fa: F[A])(f: A => G[B], g: A => H[B])
                       (G: Applicative[G], H: Applicative[H]):
                       (G[F[B]], H[F[B]])
```

12.7.5 Nested traversals

Not only can we use composed applicative functors to fuse traversals, traversable functors themselves compose. If we have a nested structure like `Map[K,Option[List[V]]]`,